

LoRaTrap: Fototrappola a lungo raggio per il monitoraggio ambientale

Leonardo Bigelli^{1*}, Emanuele Lattanzi²

Sommario

Il monitoraggio di ambienti isolati o aree protette rappresenta una notevole sfida in termini di invio di dati e accessibilità della rete. In un'ambiente isolato è importante gestire anche i falsi allarmi, in quanto spesso la raggiungibilità della zona soggetta a controllo non è facilmente raggiungibile. Il seguente progetto implementa un sistema basato su fototrappola che comunica con un protocollo LoRa, ideato per lunghe distanze e la dove non è presente la copertura internet. Inoltre è impiegato un modello di Deep Learning per l'object detection, tramite il quale è possibile effettuare la rilevazione degli oggetti all'interno dell'immagine scattata dalla fototrappola.

Keywords

LoRa — IoT — Coral — ESP32 — Deep Learning — Object Detection

¹Laurea Magistrale in Informatica Applicata, Università degli Studi di Urbino Carlo Bo, Urbino, Italia

²Docente di Programmazione per l'Internet of Things, Università degli Studi di Urbino Carlo Bo, Urbino, Italia

*Corresponding author: l.bigelli2@campus.uniurb.it

Introduzione

Il monitoraggio remoto della fauna selvatica, delle aree protette o di ambienti isolati presenta notevoli sfide in termini di consumo energetico, accessibilità alla rete e capacità di acquisire dati significativi in tempo reale. Le fototrappole tradizionali, sebbene efficaci nella rilevazione visiva, sono spesso limitate dalla necessità di interventi manuali per la raccolta delle immagini e dalla dipendenza da reti di comunicazione a corto raggio.

In questo contesto, il presente lavoro propone lo sviluppo di un sistema integrato di fototrappola intelligente, capace di rilevare eventi attraverso un sensore di movimento (PIR), acquisire immagini mediante una camera USB ad infrarossi, ed effettuare la trasmissione del risultato dell'inferenza tramite protocollo LoRa (Long Range). Questo approccio consente il monitoraggio continuo anche in ambienti remoti o privi di connettività tradizionale, garantendo al contempo efficienza energetica e copertura a lungo raggio.

L'inferenza automatica delle immagini, effettuata localmente attraverso una piattaforma Coral con accelerazione hardware, consente di identificare oggetti o soggetti d'interesse (ad esempio esseri umani o animali), riducendo drasticamente la quantità di dati da trasmettere e ottimizzando l'uso della banda disponibile. I dati ricevuti su un dispositivo di raccolta vengono successivamente archiviati o inoltrati verso sistemi di visualizzazione e analisi, come InfluxDB.

Il progetto unisce quindi computer vision, comunicazione

wireless a lungo raggio e architettura edge computing, offrendo una soluzione versatile e scalabile per applicazioni ambientali, di sicurezza e di monitoraggio.

1. Setup

In questa sezione verranno spiegati nel dettaglio tutti i componenti impiegati per lo sviluppo del seguente progetto.

1.1 Fototrappola

La fototrappola è stata pensata per scattare una fotografia solamente in caso di presenza di un generico movimento, questo per risparmiare eventuali consumi energetici evitabili. Lo scatto poi verrà processato da un modello di Deep Learning per identificare quali sono i soggetti all'interno del frame. Una volta elaborati i risultati dell'inferenza, quest'ultimi dovranno essere inviati tramite un protocollo a onde radio chiamato LoRa. Di seguito, verranno descritti tutti i componenti impiegati:

1. **Coral Dev Board:** componente fondamentale che va a definire la fototrappola in se. Esso è una piattaforma sviluppata da Google, con la peculiarità di poter eseguire direttamente le inferenze di Machine Learning in locale. Il tutto è possibile in quanto è dotata di un coprocessore Edge TPU, che permette di eseguire modelli di TensorFlow Lite in maniera efficiente e a basso consumo energetico. Nel progetto la Coral Dev Board è impiegata per elaborare le immagini in locale, utilizzando un modello pre-addestrato;
2. **Sensore PIR:** permette di rilevare il movimento da sorgenti di calore, quali animali o persone, che sono

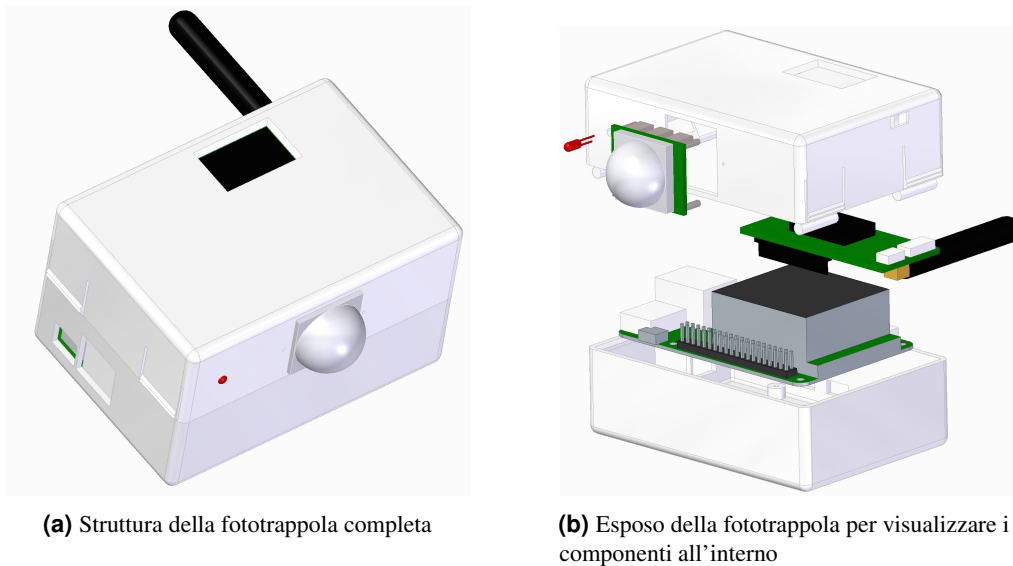


Figura 1. Rappresentazioni della struttura raffigurante la fototrappola finale. A sinistra, l'immagine della struttura finita e pronta ad essere impiegata della fototrappola. A destra, l'esposto della struttura in se, per visualizzare simbolicamente, la Coral Dev Board, l'antenna del modulo LoRa e l'ESP32.

presenti nel suo campo visivo. Esso è collegato alla Coral Dev Board, inviando un input a quest'ultima per scattare la foto, solamente quando viene rilevato un movimento. Il componente in questione contribuisce a limitare il numero di immagini scattate e quindi di conseguenza anche il numero di inferenze eseguite;

3. **Camera USB a infrarossi (IR):** dispositivo capace di catturare immagini anche in condizioni di bassa luminosità o buio totale, grazie alla presenza di LED IR e alla capacità di rilevare la luce infrarossa riflessa;
4. **ESP32:** microcontrollore potente e versatile, dotato di connettività WiFi. Nel seguente progetto vengono impiegati due di questi componenti: come trasmettitore di dati e ricevitore;
5. **Modulo LoRa:** modulo basato sul chip SX127x che permette una comunicazione wireless a lungo raggio con un consumo energetico molto ridotto. La scelta dell'impiego di questo protocollo è dovuta alla assenza di rete WiFi nella zona e moduli per connessioni LTE. Sono stati impiegati due moduli, uno per ciascun ESP32;
6. **LED:** dispositivo che emette un segnale luminoso. Impiegato per segnalare quando è stato rilevato un movimento.

La Tabella 1 descrive tutti i componenti impiegati e i collegamenti effettuati per collegarli alla Coral Dev Board. Mentre la Figura 1 illustra la fototrappola finale realizzata per essere poi posizionata in esterno.

1.2 Ricevitore

Come definito in precedenza, la fototrappola non possiede l'accesso alla rete, per questo motivo si sfrutta il modulo LoRa per comunicare i risultati ottenuti a un dispositivo che ne possiede l'accesso.

Quest'ultimo è un ESP32 con un modulo LoRa pre-installato che fungerà da ricevitore. Esso è situato in una zona in cui è presente la connessione WiFi, che sfrutterà per l'invio dei risultati a un server InfluxDb.

1.3 Modello Deep Learning

La pagina internet ufficiale della Coral Dev Board, fornisce diversi modelli pre-addestrati per quanto riguarda l'identificazione degli oggetti.

Per lo sviluppo di questo progetto è stato scelto di impiegare il modello **MobileNet SSD v2**, ottimizzato per dispositivi embedded. Il modello in questione è stato addestrato, in precedenza, su un dataset chiamato COCO. Esso contiene 80 classi di oggetti comuni, tra cui veicoli, animali, persone e oggetti vari della vita quotidiana. Per questo motivo si presta egregiamente per il compito di Object Detection per un dispositivo che deve emulare una fototrappola.

Il modello scaricabile dal sito ufficiale, è già convertito in formata TensorFlow Lite e quantizzato. Questa caratteristica è fondamentale in quanto la Coral può eseguire solamente dei modelli convertiti in TensorFlow Lite. In input il modello deve ricevere un'immagine nel formato (1, 300, 300, 3) che corrisponde ad un frame RGB ridimensionata a 300x300 pixel. In caso di immagini con formato diverso è possibile utilizzare la shape del modello in se per cambiare quella dell'immagine, rendendola compatibile. Una volta effettuata l'inferenza, il modello restituisce:

Tabella 1. Collegamenti tra Coral Dev Board e componenti esterni

Componente	Pin componente	Funzione	Pin Coral Dev Board	Note / Standard
PIR	VCC	Alimentazione	Pin 1	+3V
	GND	Ground	Pin 6	Ground
	OUT	Segnale uscita	Pin 13 (GPIO6)	I/O
ESP32	GPIO4	TX	Pin 10	UART
	GPIO25	RX	Pin 7	UART
	GND	Ground	Pin 39	Ground
LED	Anodo (+)	Uscita LED	Pin 16 (GPIO16)	I/O
	Catodo (-)	Ground	Pin 14	Ground

- Bounding box: le coordinate normalizzate dei riquadri di tutti gli oggetti identificati;
- Confidence scores: il punteggio di affidabilità per ciascun rilevamento;
- Class: la classe corrispondente inferita.

Per poter essere usato correttamente con il coprocessore Edge TPU, di cui la Coral Dev Board è fornita, il modello deve essere compilato tramite *edgetpu_compiler*.

1.4 LoRa

Il protocollo in questione, LoRa (Long Range) è una tecnologia di comunicazione wireless a bassa potenza e ideata per le comunicazioni a lungo raggio. Essa è nota ma non abbastanza discussa in ambito IoT. A differenza di altri protocolli di rete, quali WiFi o Bluetooth, LoRa utilizza una modulazione a spettro espanso chirped (CSS). Tale caratteristica dovrebbe garantire una notevole robustezza alle interferenze e una maggiore sensibilità del ricevitore. Questo permette ai dispositivi di operare in ambienti difficili.

Nel progetto si è optato per questo protocollo di comunicazione, in quanto una fototrappola è spesso posizionata in una zona dove non è presente copertura di rete. Nel caso in particolare, la posizione di essa non era raggiungibile dalla rete WiFi.

Nel dettaglio, l'implementazione del modulo LoRa è stata realizzata impiegando un ESP32 munito di LoRa SX1276 che lavora ad una frequenza di 868Hz. Per il corretto funzionamento del modulo, è necessario collegare i seguenti pin:

- MOSI (Master Output Slave Input): uscita del microcontrollore (master) e ingresso al modulo LoRa (slave), permettendo al microcontrollore stesso di inviare i comandi. Connesso al pin 27 del ESP32;
- MISO (Master Input Slave Output): è l'ingresso dati del microcontrollore e l'uscita dati del modulo LoRa. È la linea su cui il microcontrollore riceve i dati di risposta o i dati raccolti dal modulo. Connesso al pin 19;

- SCK (Serial Clock): il microcontrollore genera un segnale di clock su questo pin per sincronizzare la trasmissione dei dati. Sia il Master che lo Slave utilizzano questo segnale per campionare i bit di dati sulle linee MOSI e MISO al momento giusto. Collegato al pin 5;
- NSS (Slave Select): anche noto come CS (Chip Select), questo pin viene utilizzato dal microcontrollore per selezionare il modulo LoRa con cui vuole comunicare. Connesso al pin 18 del ESP32;
- DIO0 (Digital Input Output 0): pin (26) impiegato per notificare al microcontrollore che un evento importa è avvenuto, come la ricezione di un pacchetto dati;
- Reset: pin 23, utilizzato per forzare un hard reset del modulo LoRa, qual'ora fosse necessario ritornare allo stato iniziale.

2. Flusso di lavoro

La Figura 2 descrive il flusso di lavoro di tutto il sistema progettato. La fototrappola, tramite il sensore PIR, rileva i movimenti sia durante le ore diurne che notturne. L'accensione di un piccolo LED permette di capire che un movimento è stato rilevato. Utilizzato solo ai fini di Debug. La luce emessa è minima, in maniera tale da non sorprendere e spaventare eventuali animali. Appena il sensore PIR rileva un movimento, invia un segnale I/O alla Coral che successivamente scatterà una foto tramite l'utilizzo della camera USB IR ad essa collegata. Il frame viene poi processato dalla Coral, tramite script scritto in Python, per formattare la foto correttamente all'input che il modello di Deep Learning si aspetta.

Successivamente viene effettuata l'inferenza e il risultato di quest'ultima è trasferito al dispositivo ESP32 connessa alla Coral, tramite standard UART. L'ESP32 è munito di modulo LoRa che lavorando sulla frequenza 868MHz, invia solamente l'oggetto rilevato dalla fototrappola.

Il ricevitore ha il compito di stare perennemente in ascolto sulla stessa frequenza che il trasmettitore ha impiegato per inviare il dato. Non appena viene ricevuto il messaggio dal

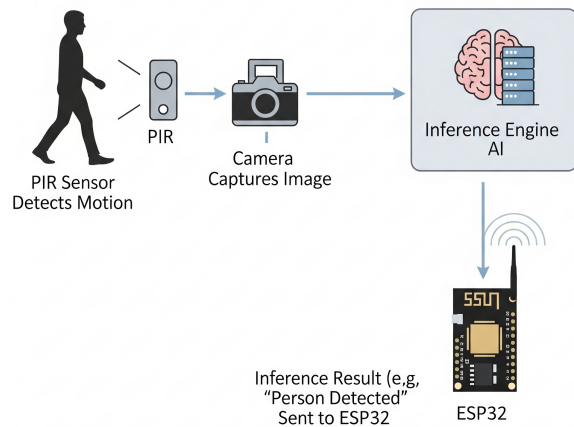


Figura 2. Illustrazione del flusso di lavoro. Il sensore PIR rileva il movimento e abilita la fotocamera di scattare una foto. Quest'ultima viene processata da un modello di Deep Learning per poi essere inviata a un ESP32 connesso alla rete.

ricevitore, quest'ultimo codifica la stringa (risultato dell'inferenza) in un indice rappresentante univocamente una classe e invia il risultato in un server InfluxDB. Al dato, prima di essere inviato, vengono associati i seguenti tag:

- host: proprietario;
- location: città in cui è posizionata la fototrappola;
- room: ambiente più preciso inerente alla posizione, permette maggior chiarezza su quale dispositivo ha inviato un dato, nel caso più di un dispositivo fosse presente.

La Tabella 2 mostra le codifiche usate dal ricevitore. Il dataset COCO possiede al suo interno 80 classi ma per l'ambiente dove è stata posizionata la fototrappola, non vengono osservate tutte le classi presenti dal dataset, ma solamente 7.

3. Raccolta dati

Il dispositivo realizzato è stato posizionato sopra un cancello principale che si affaccia sull'unica strada d'accesso, come illustrato dalla Figura 3. La scelta di questa posizione era dovuta alla necessità di raccogliere dati in un ambiente più o meno movimentato. Ovviamente non avendo posizionato la fototrappola in un ambiente boschivo, la presenza di animali selvatici è molto limitata. Il dispositivo doveva comunque essere connesso all'alimentazione limitando quindi a posizionarlo in luoghi raggiungibili da quest'ultima.

La raccolta dati è avvenuta da maggio a luglio. In questi mesi, sono state effettuate quasi 20 mila rilevazioni.

Per il primo mese, la fototrappola non era in grado di inferire

Tabella 2. Codifica da indice univoco a classe

Indice	Classe
0	Persona
1	Bicicletta
2	Macchina
3	Moto
6	Treno
13	Staccionata
14	Uccello
29	frisbee
59	Letto
62	Tv
-1	Errore generico

gli oggetti, fonte di movimento, durante le ore notturne per via della mancanza di una camera usb a infrarossi. Una volta ovviato a questo problema, essa è rimasta posizionata per tutto il periodo sopra descritto, a meno di piogge. In caso di quest'ultimo evento, il dispositivo è stato rimosso per evitare danneggiamenti.



Figura 3. Posizione della fototrappola durante tutto il periodo di campionamento. Il dispositivo è evidenziato con un cerchio rosso.

4. Analisi dei dati

Una volta completato il periodo di campionamento dei dati, sono state effettuate delle analisi. I dati in questione, sono

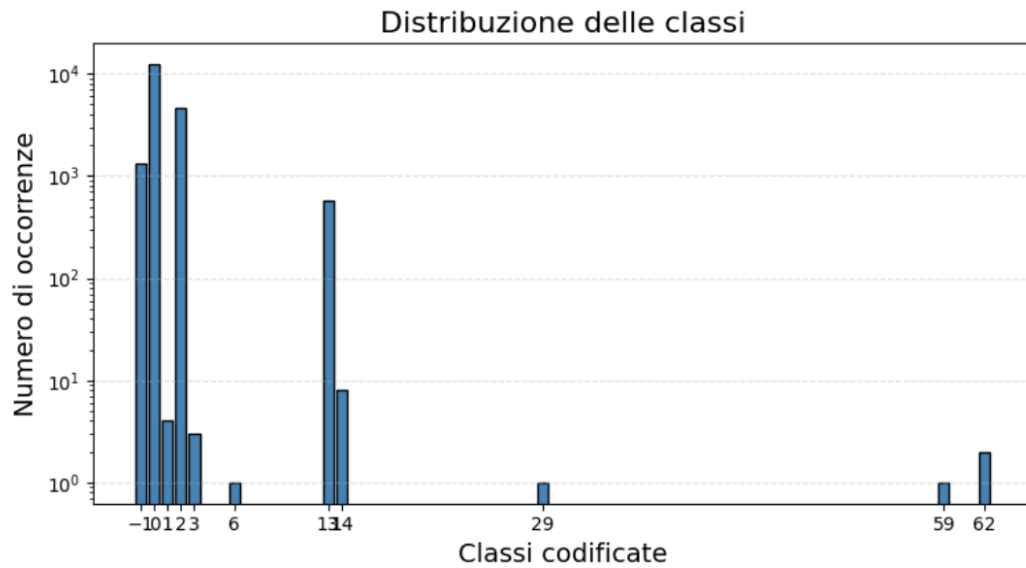


Figura 4. Distribuzioni delle classi codificate, il corrispondente oggetto è codificato nella Tabella 2. Si noti, come la classe '-1' sia molto presente.

stati ricavati tramite una query al server InfluxDB e poi gestiti con codice scritto in Python.

La Figura 5 mostra tutti i dati ottenuti durante i due mesi di rilevazione. Quest'ultimi sono codificati in classi, come descritto in precedenza. Dall'immagine, si può notare che il numero delle classi uniche presenti sono molto poche rispetto a tutto il dataset COCO, su cui il modello è stato addestrato. Nel dettaglio sono stati identificati solo 11 classi di oggetti diversi in questo periodo, con un totale di 18922 rilevazioni.

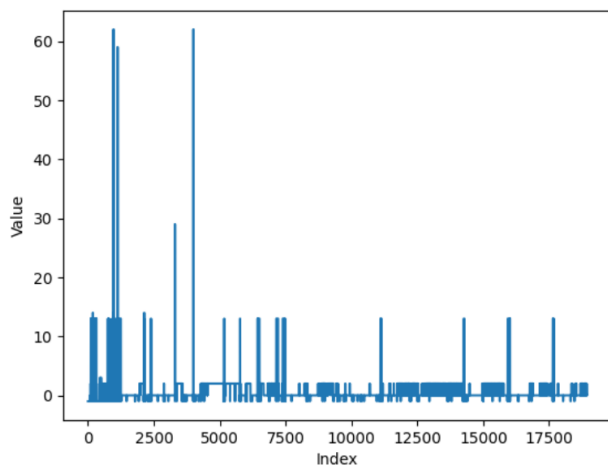


Figura 5. Tutte le rilevazioni, espresse in codici, effettuate dalla fototrappola durante il periodo della durata dell'esperimento.

Per comprendere meglio, la distribuzione delle classi inferite è stato realizzato un istogramma illustrato nella Figura 4. In quest'ultimo si può notare che le classi maggiormente presenti sono l'indice 0, -1 e 2. Da ciò si deduce che in quel periodo, il maggior flusso di movimento è stato definito dal

passare di persone e di macchine e considerando che la fototrappola è stata posizionata davanti alla strada principale, i dati ottenuti sono coerenti. La terza classe più presente è quella codificata con l'indice '-1'. Quest'ultima non è presente non dataset COCO, ma si è deciso di aggiungerla in caso di errori di trasmissione dovuti a qualunque tipo di interferenza del protocollo LoRa. Infatti, durante il lasso di tempo, è stata effettuata un'analisi dei dati in arrivo al ricevitore. Quest'ultima ha portato a conclusione che questo genere di protocollo non è molto resiliente alle interferenze. Per visualizzare meglio ciò, la Figura 6 mostra un grafico a torta rappresentante tutte le varie classi con relativa percentuale.

Come si può notare, si tutte le rilevazioni effettuate, l'errore è circa del 7%. Inizialmente il ricevitore era stato posizionato dietro ad una parete abbastanza spessa e questo ha causato molte interferenze. Successivamente, sono state provate diverse posizioni del ricevitore, fino ad arrivare alla posizione ottimale, ovvero a 15 metri dalla fototrappola, ma senza nessuno ostacolo in linea d'aria tra trasmettitore e ricevitore.

Le interferenze potrebbero essere dovute a questi fattori:

- bassa potenza di trasmissione del segnale: le antenna del modulo LoRa, sia di trasmissione che di ricezione, non sono molto grandi. Questo potrebbe implicare che la potenza di trasmissione non sia proprio all'altezza delle aspettative;
- ostacoli nel percorso: tra il trasmettitore e ricevitore sono presenti numerosi ostacoli noti per le loro capacità di schermare il segnale, quali alberi con un fogliame importante e muri spessi.

Una volta risolti questi problemi, la trasmissione è diventata abbastanza stabile.

Per quanto riguarda le classi inerenti a treni, letti o tv, sono

Distribuzione Percentuale dei Valori

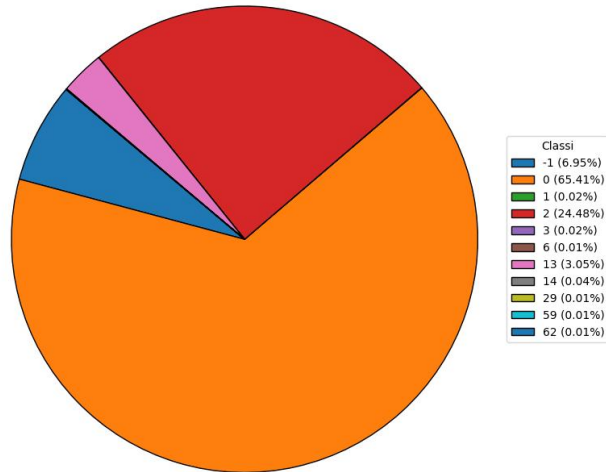


Figura 6. Grafico a torta rappresentante la percentuale di occorrenza di una singola classe su tutte le rilevazioni effettuate nel periodo di tempo.

dovuti a un errata inferenza da parte del modello di Deep Learning.

5. Limitazioni e possibili implementazioni future

Il sistema fototrappola realizzato presenta una limitazione non indifferente, ovvero il fatto di dover essere connessa sempre all'alimentazione. Questa scelta è stata presa per motivi per cui, l'architettura è stata installata in un ambiente idoneo e non si sarebbe mai spostata. Una possibile soluzione è sicuramente l'impiego di una batteria alimentata a pannelli solari, con conseguenza di dover gestire in maniera efficiente il consumo della Coral Dev Board. Quest'ultimo fattore non è banale, in quanto non sono presenti modalità di ibernazione avanzate.

Un altro problema riscontrato è sicuramente l'affidabilità del protocollo LoRa. Esso infatti è troppo soggetto a interferenze. Una possibile soluzione potrebbe essere quella di impiegare meccanismi già noti di controllo degli errori, come il CRC.

6. Conclusioni

Il seguente progetto ha lo scopo di sviluppare un sistema di Object Detection basato su fototrappola. Quest'ultima per motivi di privacy, non salva le foto in locale, ma effettua l'inferenza tramite un modello di Deep Learning, e invia il risultato a un dispositivo a sua volta connesso a internet, tramite protocollo LoRa.

Il sistema è pensato per essere adatto per ambienti in cui non è presente la connessione internet, quindi locali dispersi nella

natura, la cui trasmissione è fondamentale che non consumi un quantitativo di energia dispendioso.

Il dispositivo realizzato, rileva il movimento, scatta la foto e invia il risultato dell'inferenza a un dispositivo ESP32. Quest'ultimo carica i dati in un server InfluxDB, per avere uno storico delle rilevazioni effettuate.

L'analisi dei dati effettuata, mostra che il sistema realizzato è funzionante ma mette in risalto e in discussione l'uso del protocollo di comunicazione LoRa, in quanto non resistente alle interferenze.